

CODED — Cloud-Optimized Distributed Earth Data

Is **IPFS** a viable resilience layer for cloud-optimized geoscience data? Decomposing the cost of decentralized access (Zarr · Icechunk · xarray)

Cory Levinson · Rich Signell · Brian Terry

★ An ESIP Funding Friday Project

Objective

The geoscience community has converged on **cloud-optimized formats** — Zarr, and now **Icechunk** — that stream huge datasets via chunk-level range requests. **IPFS** offers content-addressed, location-independent, takedown-resistant storage. Can the two be married?

This round we go past "does it work" to "**where does the time actually go**" — separating **discovery**, **transfer**, and **caching costs**, and asking whether **Icechunk can read straight from IPFS with zero adapter code**.

Icechunk reads IPFS — no adapter

Icechunk 2.0.5 ships `http_storage(base_url=...)`, a read-only store over HTTP. Point it at an IPFS gateway path and the entire repo opens in xarray — the on-disk layout survives `ipfs add -r` untouched.

```
# whole Icechunk repo, straight from a CID
st = icechunk.http_storage("https://gw/ipfs/<CID>")
repo = icechunk.Repository.open(st)
sess = repo.readonly_session("main")
ds = xr.open_zarr(sess.store)
```

Old CID → old snapshot (reproducibility). Edit one of four chunks → **6 shared blocks reused**, only the changed chunk is new (cross-version dedup for free).

Caching is the real win

Once a block has been fetched, the second read serves from the local blockstore — no network round-trip. Cold→warm gains were dramatic on the long link.

6.3×

cold→warm speedup
(cross-region)

~190

MB/s warm floor,
everywhere

Warm reads converge to ~11 s / ~190 MB/s regardless of topology. A local pin **is** the warm floor — provably local (peer disconnected before read).

Where does the time go?

We read the **same 2.08 GB ERA5** 2-m temperature slice (500 Zarr chunks) three ways, in two network topologies, splitting each read into **discovery** vs **transfer**. Every path returns a bit-identical mean = **286.5062 K**.

Cold read path	Same-region	Cross-region
Local pin (warm floor)	~11 s	~11 s
Direct peer (Bitswap)	+16 s	+57 s
DHT discovery vs direct	~noise	~noise
Icechunk on S3 (control)	51 s	51 s

Transfer overhead = direct – local. Same-region ≈130 MB/s VPC; cross-region ≈36 MB/s WAN. Discovery (DHT vs direct) is within noise in a 2-node swarm — a lower-bound caveat, not a general claim.

≈0

discovery overhead
(2-node swarm, cold)

+57s

cross-region transfer
penalty vs local

What This Means

- **Blame the distance, not the DHT.** Discovery is cheap; high-RTT, single-peer transfer is the cost. More peers & regional gateways close the gap.
- **Warm ≈ free.** Caching / pinning gives repeat readers a local-speed floor — the natural pattern for shared community data.
- **Icechunk-on-IPFS is real today** via `http_storage` — reproducible, dedup-friendly, zero adapter code.

Bottom Line

Distance dominates — not discovery. Finding content on IPFS is cheap; moving 2 GB across a continent is what costs you. And the second read is nearly free.

Co-locate a gateway (or pin locally) and IPFS delivers S3-class throughput with verifiable, takedown-resistant data. Icechunk-on-IPFS needs **no custom code**.